Solutions des TD : Bases des systèmes d'exploitation

Mai 2025

TD1 : Processus et Threads

Résumé : Ce TD se concentre sur la compréhension des processus et des threads. Un processus est un programme en exécution, tandis qu'un thread est une tâche plus petite au sein d'un processus qui partage la mémoire. Les processus peuvent être indépendants (sans partage) ou coopératifs (partageant des données pour des tâches comme le multitâche). Nous explorerons également comment les processus sont créés avec `fork()`, comment ils s'exécutent et comment gérer leur comportement dans des programmes C.

Question 1 : Que affiche le programme suivant lorsqu'il est exécuté ?

Programme :

```
int main() {
        pid_t pid;
        int x = 1;
        pid = fork();
        if (pid == 0) {
                printf("Dans fils : x=%d\n", ++x);
                exit(0);
        }
        printf("Dans p re : x=%d\n", --x);
        exit(0);
}
```

Solution :

- **Fonctionnement de `fork()`** : `fork()` crée un nouveau processus (fils) qui est une copie du processus actuel (père). Après `fork()`, les deux processus s'exécutent indépendamment.
- **Variable `x`** : `x` commence à 1. Puisque `fork()` crée une copie, le père et le fils ont chacun leur propre `x` avec la valeur 1.
- **Comprendre `pid`** :
  - `pid = fork()` retourne 0 dans le processus fils et l'ID du processus fils dans le processus père.

- Donc, `if (pid == 0)` est vrai pour le fils, et le père exécute l'autre partie.

- **Processus Fils** (`pid == 0`) :
  - Exécute `printf("Dans fils :  x=%d", ++x)`.
  - `++x` signifie augmenter `x` de 1, puis l'utiliser. Le `x` du fils est 1, donc `++x` le fait passer à 2.
  - Affiche : `Dans fils :  x=2`.

- **Processus Père** :
  - Exécute `printf("Dans père :  x=%d", -x)`.
  - `-x` signifie diminuer `x` de 1, puis l'utiliser. Le `x` du père est 1, donc `-x` le fait passer à 0.
  - Affiche : `Dans père :  x=0`.

- **Sortie** : Les deux processus affichent leurs messages. L'ordre n'est pas garanti (dépend de l'ordonnancement), mais les sorties possibles sont :
  - `Dans fils :  x=2`
    `Dans père :  x=0`
    OU

  - `Dans père :  x=0`
    `Dans fils :  x=2`

Question 2 : Illustrez l'exécution de ce programme :

Programme :

```
int main() {
        fork();
        fork();
        fork();
        printf("hello!\n");
        exit(0);
}
```

Solution :

- **Fonctionnement de `fork()`** : Chaque `fork()` double le nombre de processus en créant un nouveau processus fils.

- **Étape par étape** :
  1. **Premier `fork()`** : 1 processus devient 2 (original + 1 fils).
  2. **Deuxième `fork()`** : Les deux processus appellent `fork()`, donc 2 deviennent 4 (chaque processus crée un nouveau fils).
  3. **Troisième `fork()`** : Les 4 processus appellent `fork()`, donc 4 deviennent 8 (chaque processus crée un nouveau fils).

- **Nombre total de processus** : Après trois appels à `fork()`, il y a $2^3 = 8$ processus.
- **Actions de chaque processus** : Chacun des 8 processus exécute `printf("hello!")`.
- **Sortie** : Le programme affiche `hello!` 8 fois. L'ordre n'est pas fixe (dépend de l'ordonnancement), mais cela ressemblera à :

```
hello!
hello!
hello!
hello!
hello!
hello!
hello!
hello!
```

Question 3 (TD/TP) : Écrire un programme C qui crée deux processus fils : l'un affiche les entiers de 1 à 5, et l'autre affiche les entiers de 6 à 10. Définissez un intervalle de 1 seconde entre chaque entier affiché.

Solution :

- **Objectif** : Créer deux processus fils avec `fork()`. Le premier affiche de 1 à 5, le second de 6 à 10, avec un délai de 1 seconde entre chaque nombre.
- **Démarche** :
  1. Utiliser `fork()` deux fois pour créer deux processus fils.
  2. Le premier fils affiche de 1 à 5, le second de 6 à 10, le père attend que les deux terminent.
  3. Utiliser `sleep(1)` pour attendre 1 seconde entre chaque nombre.
- **Programme :**

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
        pid_t pid1, pid2;

        // Cr er le premier fils
        pid1 = fork();
        if (pid1 == 0) {
                // Premier fils : affiche de 1    5
                for (int i = 1; i <= 5; i++) {
```

```
                                        printf("Fils 1 : %d\n", i);
                                        sleep(1); // Attendre 1 seconde
                                }
                                exit(0);
                        }

                        // Cr er le deuxi me fils
                        pid2 = fork();
                        if (pid2 == 0) {
                                // Deuxi me fils : affiche de 6    10
                                for (int i = 6; i <= 10; i++) {
                                        printf("Fils 2 : %d\n", i);
                                        sleep(1); // Attendre 1 seconde
                                }
                                exit(0);
                        }

                        // Le p re attend que les deux fils terminent
                        wait(NULL);
                        wait(NULL);
                        printf("P re : Les deux fils ont termin .\n");
                        return 0;
                }
```

- **Explication :**
  - `pid1 = fork()` crée le premier fils. Il affiche de 1 à 5 avec un délai de 1 seconde.
  - `pid2 = fork()` crée le deuxième fils. Il affiche de 6 à 10 avec un délai de 1 seconde.
  - `sleep(1)` met en pause pendant 1 seconde après chaque affichage.
  - `wait(NULL)` fait attendre le père jusqu'à ce que les deux fils terminent.
- **Exemple de sortie** : Les fils s'exécutent en même temps, donc les nombres peuvent s'entrelacer :

```
Fils 1 : 1
Fils 2 : 6
```

```
Fils 1 : 2
Fils 2 : 7
Fils 1 : 3
Fils 2 : 8
Fils 1 : 4
Fils 2 : 9
Fils 1 : 5
Fils 2 : 10
Père : Les deux fils ont terminé.
```

Question 4 (TD/TP) : Écrire un programme C qui incrémente une variable `i` indéfiniment et affiche `i` lorsqu'il est un multiple de 100. Utilisez `sleep` pour ralentir l'exécution. Que remarquez-vous lorsque vous l'exécutez plusieurs fois ?

Solution :

- **Objectif** : Incrémenter `i` à l'infini, afficher lorsque `i` est un multiple de 100, et ralentir la boucle avec `sleep`.
- **Démarche** :
  1. Utiliser une boucle `while(1)` pour exécuter indéfiniment.
  2. Vérifier si `i` est un multiple de 100 avec `i % 100 == 0`.
  3. Utiliser `sleep(1)` pour ralentir la boucle.
- **Programme :**

```c
#include <stdio.h>
#include <unistd.h>

int main() {
        int i = 0;
        while (1) {
                i++; // Incrémenter i
                if (i % 100 == 0) {
                        printf("i = %d\n", i);
                }
                sleep(1); // Attendre 1 seconde
        }
        return 0;
}
```

- **Explication :**
  - `i` commence à 0 et augmente de 1 à chaque boucle.
  - `i % 100 == 0` vérifie si `i` est un multiple de 100 (comme 100, 200, 300).

- `sleep(1)` ralentit la boucle pour éviter un affichage trop rapide.

- **Exemple de sortie :**

```
i = 100
i = 200
i = 300
...
```

- **Observation (TP)** :
  - À chaque exécution du programme, il repart de zéro (`i` est réinitialisé à 0).
  - Il affiche les multiples de 100 (100, 200, 300, etc.) toutes les secondes.
  - Si vous l'exécutez plusieurs fois (par exemple, dans différentes fenêtres de terminal), chaque instance s'exécute indépendamment, et leurs sorties peuvent se mélanger :

    ```
    i = 100  (de l'instance 1)
    i = 100  (de l'instance 2)
    i = 200  (de l'instance 1)
    i = 200  (de l'instance 2)
    ```

  - **Observation** : Les sorties s'entrelacent car chaque instance s'exécute en même temps, et le système d'exploitation les ordonnance. Cela montre comment plusieurs processus peuvent s'exécuter simultanément mais interférer avec leurs sorties.

Question 5 : Quelle est la différence entre un processus et un thread ?

Solution :

- Un **processus** est comme une grande boîte : c'est un programme en exécution avec son propre espace mémoire. Exemple : ouvrir un navigateur web crée un processus.

- Un **thread** est une tâche plus petite dans cette boîte : les threads partagent la même mémoire mais travaillent sur différentes tâches. Exemple : un thread charge une page web, un autre lit une vidéo dans le navigateur.

- **Différence clé** : Les processus ne partagent pas la mémoire (ils sont séparés), mais les threads d'un même processus partagent la mémoire, ce qui les rend plus rapides mais plus délicats à gérer.

Question 6 : Pourquoi les processus coopèrent-ils ? Donnez un exemple.

Solution :

- Les processus coopèrent pour partager des données et travailler ensemble sur des tâches.

- **Raisons :**
  1. **Partage d'informations** : Pour transmettre des données entre programmes.
  2. **Vitesse** : Diviser les tâches pour finir plus vite.
  3. **Modularité** : Décomposer les tâches en parties plus petites.
- **Exemple** : Vous éditez un document (un processus) tout en l'imprimant (un autre processus). Ils coopèrent en partageant le fichier pour que vous puissiez éditer et imprimer en même temps.

TD2 : Synchronisation des processus

Résumé : La synchronisation assure que les processus travaillent ensemble sans corrompre les données partagées. Sans elle, des conditions de course surviennent (par exemple, deux processus modifient la même variable en même temps, causant des erreurs). La synchronisation oblige les processus à attendre leur tour. Ce TD explore également les sections critiques et l'exclusion mutuelle avec l'accès aux fichiers et des solutions logicielles comme l'algorithme de Peterson.

Question 1 : Nous avons trois fichiers F1, F2 et F3, et trois processus avec les programmes A, B et C structurés comme suit :

Programmes :

- **Programme A**
  - actions A1
  - actions A2 (lecture F2)
  - actions A3
  - actions A4 (écriture F3)
  - actions A5
- **Programme B**
  - actions B1 (écriture F3)
  - actions B2
  - actions B3 (lecture F1)
  - actions B4
  - actions B5
- **Programme C**
  - actions C1 (lecture F3)
  - actions C2
  - actions C3
  - actions C4 (écriture F2)

  - actions C5

Contrainte : Chaque fichier ne peut pas être lu et modifié en même temps, ni modifié par plusieurs processus simultanément. Réécrivez le tableau ci-dessus en ajoutant des commandes entre les actions où nécessaire pour mettre en évidence les sections critiques. Déduisez les sections d'exclusion mutuelle. **Solution :**

- **Sections critiques** : Une section critique est toute action qui lit ou écrit dans un fichier (F1, F2 ou F3) car un seul processus peut accéder à un fichier à la fois (pas de chevauchement lecture/écriture ni écritures multiples). Nous utiliserons un verrou (par exemple, `lock(fichier)` et `unlock(fichier)`) pour protéger ces sections.
- **Règles :**
  - Utiliser `lock(fichier)` avant de lire ou écrire pour assurer un accès exclusif.
  - Utiliser `unlock(fichier)` après avoir terminé pour permettre aux autres d'y accéder.
- **Programmes mis à jour :**
  - **Programme A :**
    * actions A1
    * `lock(F2)`
    * actions A2 (lecture F2)
    * `unlock(F2)`
    * actions A3
    * `lock(F3)`
    * actions A4 (écriture F3)
    * `unlock(F3)`
    * actions A5
  - **Programme B :**
    * `lock(F3)`
    * actions B1 (écriture F3)
    * `unlock(F3)`
    * actions B2
    * `lock(F1)`
    * actions B3 (lecture F1)
    * `unlock(F1)`
    * actions B4
    * actions B5
  - **Programme C :**
    * `lock(F3)`
    * actions C1 (lecture F3)
    * `unlock(F3)`
    * actions C2

    * actions C3
    * `lock(F2)`
    * actions C4 (écriture F2)
    * `unlock(F2)`
    * actions C5

- **Sections d'exclusion mutuelle :**
  - **F2** : Protégé lorsque A lit (A2) et C écrit (C4). Un seul peut accéder à F2 à la fois.
  - **F3** : Protégé lorsque B écrit (B1), A écrit (A4), et C lit (C1). Un seul peut accéder à F3 à la fois.
  - **F1** : Protégé lorsque B lit (B3). Un seul peut accéder à F1 à la fois.
- **Explication :** Les verrous garantissent qu'un processus qui lit ou écrit dans un fichier bloque les autres jusqu'à ce qu'il ait terminé, évitant des conflits comme une lecture pendant une écriture ou deux écritures simultanées.

Question 2 : Solutions pour deux processus (Peterson, 1981) :

Programme :

```
#define FALSE 0
#define TRUE 1
#define N 2 /* nombre de processus */
int turn; /*    qui le tour */
int interested[N]; /* tous initialement    0 (FALSE) */
void enter_region(int process) /* process est 0 ou 1 */
{
        int other; /* num ro de  l autre processus */
        other = 1 - process; /*   l  oppos   de process */
        interested[process] = TRUE; /* Indique que vous  tes  int ress  */
        turn = process; /* d finit  l indicateur */
        while (turn == process && interested[other] == TRUE)
        ; /* instruction vide */
}
void leave_region(int process) /* process : qui sort */
{
        interested[process] = FALSE; /* indique la sortie de la section critique */
}
```

Problème : La solution de Peterson est une solution logicielle pour l'exclusion mutuelle. Expliquez cette solution, en montrant que cet algorithme résout le problème de l'exclusion mutuelle de manière équitable. Généralisez l'algorithme pour n processus. **Solution :**

- **Fonctionnement** : L'algorithme de Peterson garantit qu'un seul des deux processus (0 ou 1) entre dans sa section critique à la fois, en utilisant des variables partagées `turn` et `interested[]`.
- **Mécanisme :**
  1. **Entrer dans la section critique** (`enter`$_{r}egion$) :
     2. `interested[process] = TRUE` **: Le processus signale qu'il veut entrer.**
     3. `turn = process` **: Il revendique son tour.**
     4. `while (turn == process && interested[other] == TRUE)` **: Il attend si l'autre processus est aussi intéressé et que ce n'est pas son tour. C'est une attente courtoise—le processus cède si l'autre est intéressé.**
- **Sortir de la section critique** (`leave`$_{r}egion$) :
  - `interested[process] = FALSE` **: Le processus signale qu'il a terminé, permettant à l'autre de procéder.**

Exclusion mutuelle :

- Un seul processus peut avoir `turn` défini à son ID, et la boucle `while` garantit que l'autre attend si les deux sont intéressés. Si l'un entre, l'autre est bloqué, assurant qu'un seul est dans la section critique.

Équité :

- Si les deux processus veulent entrer, la variable `turn` décide qui va en premier. Le processus en attente continue de vérifier, et une fois que l'autre sort (met `interested` à FALSE), il obtient son tour. Cela garantit qu'aucun processus n'est affamé—c'est une approche premier arrivé, premier servi avec un bris d'égalité (`turn`).

Généralisation à n processus :

- Pour n processus, il faut un système plus complexe. Une méthode est d'utiliser une approche basée sur des tickets :
  - **Variables :**
    - `interested[n]` : Tableau indiquant l'intérêt de chaque processus (TRUE ou FALSE).
    - `turn[n][n]` : Une matrice où `turn[i][j]` est TRUE si le processus i doit attendre j.
  - **Algorithme :**
    1. Chaque processus `p` met `interested[p] = TRUE`.
    2. Met `turn[p][other] = FALSE` pour tous les autres processus `other` (il n'attend pas après eux).

3. Attend tant qu'il existe un processus `other` où `interested[other] == TRUE` et `turn[other][p] == FALSE` (ce qui signifie que `other` est intéressé et que p doit attendre).
4. Lorsqu'il a terminé, met `interested[p] = FALSE`.

– **Explication :** Cela assure l'exclusion mutuelle en obligeant chaque processus à attendre que tous les autres terminent leurs sections critiques, généralisant l'équité de la méthode de Peterson. La matrice `turn` gère les dépendances, garantissant qu'aucun processus n'entre simultanément.

Question 3 : Considérons le problème Producteur-Consommateur vu en cours. n producteurs génèrent des messages et les placent dans un tampon illimité. n consommateurs peuvent récupérer des messages du tampon.

Programme :

```
Semaphore Mutex;
Message tampon[];

Producteur() {
        Message m;
        Tantque Vrai faire
        m = creermessage();
        wait(Mutex);
        Ecriture Tampon(m);
        signal(Mutex);
        Fin Tantque
}

Consommateur() {
        Message m;
        Tantque Vrai faire
        wait(Mutex);
        m = Lecture Tampon()
        signal(Mutex);
        Fin Tantque
}
```

Problème : Adaptez cette solution pour garantir que l'échange de messages est synchronisé (les consommateurs sont bloqués si le tampon est vide). **Solution :**

- **Problème** : La solution actuelle utilise un seul `Mutex` pour empêcher plusieurs producteurs ou consommateurs d'accéder au tampon simultanément, mais elle ne bloque pas les consommateurs lorsque le tampon est vide.
- **Correction** : Ajouter un sémaphore `full` pour suivre les messages dans le tampon. Les consommateurs attendent sur `full` si c'est 0 (tampon vide).
- **Solution mise à jour :**
  – **Variables :**

    - `Semaphore Mutex` (valeur initiale 1) : Contrôle l'accès au tampon.
    - `Semaphore full` (valeur initiale 0) : Compte les messages dans le tampon ; les consommateurs attendent si 0.
  - **Producteur()** :
    - `Message m;`
    - `Tantque Vrai faire`
      - `m = creermessage();`
      - `wait(Mutex);`
      - `Ecriture Tampon(m);`
      - `signal(Mutex);`
      - `signal(full);` // Signale qu'un message est ajouté
    - `Fin Tantque`
  - **Consommateur()** :
    - `Message m;`
    - `Tantque Vrai faire`
      - `wait(full);` // Attend si le tampon est vide
      - `wait(Mutex);`
      - `m = Lecture Tampon();`
      - `signal(Mutex);`
    - `Fin Tantque`

- **Explication :**
  - `full` commence à 0 (aucun message). Les producteurs ajoutent un message et signalent `full`, l'augmentant.
  - Les consommateurs attendent sur `full` si c'est 0, bloquant jusqu'à ce qu'un producteur ajoute un message.
  - `Mutex` empêche toujours les accès multiples, garantissant qu'un seul processus modifie le tampon à la fois.

Question 4 : Maintenant, considérons un tampon de taille Max (il peut contenir au maximum Max messages). Ajoutez les modifications nécessaires pour prendre en compte cette nouvelle contrainte (les producteurs sont bloqués si le tampon est plein).

Solution :

- **Problème** : Avec un tampon limité (taille Max), les producteurs doivent attendre si le tampon est plein.
- **Correction** : Ajouter un sémaphore `empty` pour suivre les emplacements disponibles. Les producteurs attendent sur `empty` si c'est 0 (tampon plein).
- **Solution mise à jour :**

- **Variables :**
  * `Semaphore Mutex` (valeur initiale 1) : Contrôle l'accès au tampon.
  * `Semaphore full` (valeur initiale 0) : Compte les messages dans le tampon.
  * `Semaphore empty` (valeur initiale Max) : Compte les emplacements vides.
- **Producteur()** :
  * `Message m;`
  * `Tantque Vrai faire`
    · `m = creermessage();`
    · `wait(empty);` // Attend si le tampon est plein
    · `wait(Mutex);`
    · `Ecriture Tampon(m);`
    · `signal(Mutex);`
    · `signal(full);` // Signale qu'un message est ajouté
  * `Fin Tantque`
- **Consommateur()** :
  * `Message m;`
  * `Tantque Vrai faire`
    · `wait(full);` // Attend si le tampon est vide
    · `wait(Mutex);`
    · `m = Lecture Tampon();`
    · `signal(Mutex);`
    · `signal(empty);` // Signale qu'un emplacement est libéré
  * `Fin Tantque`

- **Explication :**
  - `empty` commence à Max (tous les emplacements libres). Les producteurs attendent sur `empty` si c'est 0 (plein).
  - Les consommateurs libèrent un emplacement avec `signal(empty)` après lecture.
  - Cela équilibre producteurs et consommateurs, bloquant les producteurs quand c'est plein et les consommateurs quand c'est vide.

Question 5 : Cinq philosophes sont assis autour d'une table circulaire, passant leur temps à manger et à penser. La table a cinq fourchettes (une par place) placées entre chaque philosophe. Le repas est composé de spaghettis, qui, selon leur coutume, nécessitent deux fourchettes pour manger.

Problème :

- Chaque philosophe prend une place.

- Un philosophe qui mange utilise la fourchette à sa droite et à sa gauche (pas celle en face).
- À tout moment, chaque philosophe est dans un état :
  - Mange avec deux fourchettes.
  - A faim, attend la fourchette droite, gauche, ou les deux.
  - Ne peut pas prendre une fourchette tenue par un voisin.
  - A fini de manger, repose les deux fourchettes et recommence à penser.
  - Pense, n'utilise pas de fourchettes.
- Initialement, tous les philosophes pensent.
- Un philosophe arrête de manger en un temps limité.
- **Question 1 :** Montrez qu'il y a un problème d'interblocage ; proposez une solution.
- **Question 2 :** Ajoutez l'hypothèse : les philosophes sont droitiers—ils ne prennent pas la fourchette gauche sans avoir d'abord la fourchette droite. Proposez une nouvelle solution.

Solution :

- **Question 1 : Problème d'interblocage** :
  - **Comment l'interblocage se produit** : Si les cinq philosophes prennent chacun leur fourchette droite en même temps (par exemple, le Philosophe 1 prend la fourchette 1, le Philosophe 2 prend la fourchette 2, etc.), chacun attend la fourchette gauche (tenue par le voisin). Cela crée une attente circulaire (1 attend 2, 2 attend 3, ..., 5 attend 1), provoquant un interblocage.
  - **Solution** : Utiliser un sémaphore pour chaque fourchette et une règle pour briser le cycle :
    - **Variables** : `Semaphore fork[5]` (valeur initiale 1 pour chaque fourchette).
    - **Actions du Philosophe i** :
      - `wait(fork[i])` // Prendre la fourchette droite
      - `wait(fork[(i+1) % 5])` // Prendre la fourchette gauche
      - Manger (temps limité)
      - `signal(fork[(i+1) % 5])` // Relâcher la fourchette gauche
      - `signal(fork[i])` // Relâcher la fourchette droite
      - Penser
    - **Amélioration** : Ajouter un `Semaphore room` global (valeur initiale 4) pour limiter à quatre philosophes mangeant simultanément (un doit attendre, brisant le cycle).
      - Avant de prendre les fourchettes : `wait(room)`.
      - Après avoir relâché les fourchettes : `signal(room)`.
  - **Explication** : Limiter à quatre garantit qu'au moins un philosophe attend, empêchant tous d'entrer dans l'état d'interblocage.

- **Question 2 : Solution pour droitiers** :
  - **Nouvelle règle** : Les philosophes droitiers prennent d'abord la fourchette droite, puis la gauche.
  - **Solution** : Ajuster l'ordre de prise des fourchettes :
    - **Actions du Philosophe i** :
      - `wait(fork[i])` // Prendre la fourchette droite
      - `wait(fork[(i+1) % 5])` // Prendre la fourchette gauche
      - Manger (temps limité)
      - `signal(fork[(i+1) % 5])` // Relâcher la fourchette gauche
      - `signal(fork[i])` // Relâcher la fourchette droite
      - Penser
  - **Pourquoi ça marche** : Puisque tous prennent la fourchette droite en premier, un seul philosophe (par exemple, le dernier à essayer) trouvera la fourchette gauche prise, l'obligeant à attendre. Cela brise l'attente circulaire, car tous ne peuvent pas tenir une fourchette droite simultanément. Aucun sémaphore supplémentaire n'est nécessaire si l'ordre est strictement suivi.
  - **Explication** : La règle de la main droite crée une dépendance linéaire (par exemple, 1 attend la gauche de 2, 2 attend la gauche de 3, etc.), évitant un cycle.

Question 6 : Cinq philosophes sont représentés avec des états :

Programme :

```
enum {THINKING, HUNGRY, EATING} state[5];
Philosopher i can set the variable state[i] = EATING only if her two neighbors are not eating :
(state[(i+4) % 5] != EATING) and (state[(i+1) % 5] != EATING).
We also need to declare condition self[5];
monitor DiningPhilosophers {
        enum {THINKING, HUNGRY, EATING} state[5];
        condition self[5];
        void pickup(int i) {
                state[i] = HUNGRY;
                test(i);
                if (state[i] != EATING) self[i].wait();
        }
        void putdown(int i) {
                state[i] = THINKING;
                test((i + 4) % 5);
                test((i + 1) % 5);
        }
        void test(int i) {
```

```
                                if ((state[(i + 4) % 5] != EATING)
                                    && (state[i] == HUNGRY) && (
                                   state[(i + 1) % 5] != EATING))
                                   {
                                        state[i] = EATING;
                                        self[i].signal();
                                }
                        }
                        initialization_code() {
                                for (int i = 0; i < 5; i++)
                                state[i] = THINKING;
                        }
                }
```

Solution :

- **Fonctionnement** : Cette solution basée sur un moniteur utilise un tableau partagé `state` et des variables de condition pour gérer le problème des philosophes avec exclusion mutuelle.
- **Mécanisme :**
  - **pickup(i)** :
    - Met `state[i] = HUNGRY` pour montrer l'intention.
    - Appelle `test(i)` pour vérifier si i peut manger (voisins ne mangent pas).
    - Si pas en train de manger, `self[i].wait()` bloque jusqu'à ce qu'il soit signalé.
  - **putdown(i)** :
    - Met `state[i] = THINKING` après avoir mangé.
    - Appelle `test()` sur les deux voisins pour voir s'ils peuvent manger maintenant.
  - **test(i)** :
    - Vérifie si i a faim et si les deux voisins ne mangent pas.
    - Si vrai, met `state[i] = EATING` et signale `self[i]` pour réveiller le philosophe en attente.
  - **Initialisation** : Tous commencent à penser.
- **Exclusion mutuelle** : Le moniteur garantit qu'un seul philosophe modifie `state` ou signale à la fois. La condition `test()` empêche de manger si les voisins mangent, évitant les conflits.
- **Explication** : Cette solution est équitable car un philosophe affamé attend seulement s'il est bloqué par ses voisins, et `putdown()` vérifie les voisins, leur permettant de procéder quand possible. Elle prévient l'interblocage en garantissant que tous ne peuvent pas manger simultanément.

Question 7 : Qu'est-ce qu'une section critique, et pourquoi avons-nous besoin de l'exclusion mutuelle ?

Solution :

- Une **section critique** est une partie du code où un processus utilise une ressource partagée, comme une variable ou un fichier.
- **Exemple** : Deux processus mettent à jour un solde bancaire (variable partagée). S'ils mettent à jour en même temps, le solde pourrait être erroné.
- **Exclusion mutuelle** signifie qu'un seul processus peut être dans sa section critique à la fois. C'est comme une clé de salle de bain—une seule personne peut l'utiliser à la fois pour éviter un désordre.
- **Pourquoi nécessaire** : Pour éviter des erreurs (comme dans l'exemple bancaire) en s'assurant que les processus prennent leur tour.

Question 8 : Comment pouvons-nous garantir l'exclusion mutuelle dans un programme ?

Solution :

- Utiliser un **verrou** :
  1. Avant d'entrer dans la section critique, un processus prend le verrou (comme prendre une clé).
  2. Les autres processus attendent si le verrou est pris.
  3. Après avoir terminé, le processus relâche le verrou, et un autre peut le prendre.
- **Exemple** : Deux processus imprimant sur la même imprimante. Un verrou garantit que l'un imprime d'abord, puis l'autre, pour que les pages ne se mélangent pas.

TD3 : Ordonnancement des processus

Résumé : L'ordonnancement des processus décide quel processus s'exécute et quand, pour optimiser l'utilisation du CPU. Les algorithmes comme FCFS (Premier Arrivé, Premier Servi), SJF (Tâche la plus courte d'abord) et Round Robin (rotation) sont utilisés. Ce TD explore ces algorithmes, leurs avantages, leurs inconvénients et leur application dans des scénarios pratiques.

Question 1 : Expliquez les algorithmes d'ordonnancement FCFS, SJF et Round Robin. Quels sont leurs avantages et inconvénients ?

Solution :

- **FCFS (Premier Arrivé, Premier Servi)** :

- **Fonctionnement** : Les processus sont exécutés dans l'ordre de leur arrivée. Le premier processus qui arrive est exécuté en premier, suivi du suivant, etc.
- **Avantages** :
  * Simple à implémenter.
  * Équitable dans l'ordre d'arrivée.
- **Inconvénients** :
  * Effet de convoi : Si un processus long arrive en premier, les processus courts derrière doivent attendre longtemps.
  * Temps d'attente moyen élevé si les processus varient beaucoup en durée.

- **SJF (Tâche la plus courte d'abord)** :
  - **Fonctionnement** : Parmi les processus prêts, celui avec le temps d'exécution le plus court est exécuté en premier.
  - **Avantages** :
    * Minimise le temps d'attente moyen (optimal pour cela).
    * Favorise les processus courts.
  - **Inconvénients** :
    * Nécessite de connaître le temps d'exécution à l'avance, ce qui est difficile.
    * Peut affamer les processus longs si des processus courts arrivent constamment.

- **Round Robin (Rotation)** :
  - **Fonctionnement** : Chaque processus reçoit un quantum de temps (par exemple, 10 ms). Si un processus ne termine pas dans ce quantum, il est mis en fin de file et le suivant est exécuté.
  - **Avantages** :
    * Équitable : chaque processus a sa chance régulièrement.
    * Bon pour les systèmes interactifs (temps de réponse rapide).
  - **Inconvénients** :
    * Dépend du quantum : trop court, trop de changements de contexte ; trop long, ressemble à FCFS.
    * Temps d'attente moyen plus élevé que SJF pour des processus variés.

Question 2 : Considérons quatre processus P1, P2, P3, P4 avec les temps d'arrivée et d'exécution suivants :

- P1 : Arrivée = 0, Exécution = 8
- P2 : Arrivée = 1, Exécution = 4
- P3 : Arrivée = 2, Exécution = 9
- P4 : Arrivée = 3, Exécution = 5

Calculez le temps d'attente moyen pour chaque algorithme (FCFS, SJF, Round Robin avec quantum = 4). Solution :

- **FCFS** :
  - Ordre : P1 (0-8), P2 (8-12), P3 (12-21), P4 (21-26).
  - Temps d'attente :
    * P1 : 0 (commence immédiatement).
    * P2 : 8 - 1 = 7.
    * P3 : 12 - 2 = 10.
    * P4 : 21 - 3 = 18.
  - Temps d'attente moyen : $(0 + 7 + 10 + 18)/4 = 35/4 = 8,75$.
- **SJF** :
  - À t=0 : P1 (exécute 0-8).
  - À t=8 : P2 (4), P3 (9), P4 (5). Ordre : P2 (8-12), P4 (12-17), P3 (17-26).
  - Temps d'attente :
    * P1 : 0.
    * P2 : 8 - 1 = 7.
    * P3 : 17 - 2 = 15.
    * P4 : 12 - 3 = 9.
  - Temps d'attente moyen : $(0 + 7 + 15 + 9)/4 = 31/4 = 7,75$.
- **Round Robin (quantum = 4)** :
  - t=0-4 : P1 (reste 4).
  - t=4-8 : P2 (termine à 8).
  - t=8-12 : P3 (reste 5).
  - t=12-16 : P4 (reste 1).
  - t=16-20 : P1 (termine à 20).
  - t=20-24 : P3 (reste 1).
  - t=24-25 : P4 (termine à 25).
  - t=25-26 : P3 (termine à 26).
  - Temps de début effectif (excluant les exécutions partielles) :
    * P1 : 0 (premier bloc), termine à 20.
    * P2 : 4, termine à 8.
    * P3 : 8, termine à 26.
    * P4 : 12, termine à 25.
  - Temps d'attente (temps avant le début de la première exécution) :
    * P1 : 0.
    * P2 : 4 - 1 = 3.

    - P3 : 8 - 2 = 6.
    - P4 : 12 - 3 = 9.
  - Temps d'attente moyen : $(0 + 3 + 6 + 9)/4 = 18/4 = 4,5$.

- **Conclusion** : Round Robin a le meilleur temps d'attente moyen (4,5), suivi de SJF (7,75), puis FCFS (8,75). Cela montre que Round Robin est efficace pour réduire l'attente dans ce scénario.

Question 3 : Quels sont les critères pour évaluer un algorithme d'ordonnancement ?

Solution :

- **Critères d'évaluation :**
  1. **Temps d'attente moyen** : Temps moyen qu'un processus passe en attente avant d'être exécuté.
  2. **Temps de réponse** : Temps entre l'arrivée d'un processus et son premier accès au CPU (important pour les systèmes interactifs).
  3. **Temps de traitement** : Temps total pour qu'un processus termine (attente + exécution).
  4. **Utilisation du CPU** : Pourcentage de temps où le CPU est occupé (idéalement proche de 100 %).
  5. **Équité** : Assure que tous les processus ont une chance d'être exécutés, évitant la famine.
- **Exemple** : Dans un système interactif, le temps de réponse est crucial pour que les utilisateurs ne ressentent pas de retard, tandis que dans un système batch, minimiser le temps de traitement est plus important.

TD4 : Sémaphore et Problèmes de Synchronisation

Résumé : Les sémaphores sont des outils pour gérer les ressources partagées. Un sémaphore binaire agit comme un verrou (0 ou 1), tandis qu'un sémaphore comptant gère plusieurs ressources (par exemple, 5 imprimantes). Ils sont utilisés pour résoudre des problèmes comme Producteur-Consommateur, assurant une coordination correcte. Ce TD explore la communication unidirectionnelle et bidirectionnelle avec des tubes anonymes, les tubes nommés pour client-serveur, les files de messages, et la synchronisation basée sur les signaux.

Question 1 : Communication unidirectionnelle avec les tubes anonymes

Problème : Écrire un programme C qui crée deux processus : le processus père écrit le message « Bonjour, fils. Je suis ton père ! ». Le fils le récupère, puis l'affiche en utilisant un tube anonyme. **Solution :**

- **Démarche** : Utiliser `pipe()` pour créer un tube anonyme, `fork()` pour créer un fils, et `write()`/`read()` pour envoyer/recevoir le message.

- **Programme :**

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

int main() {
        int fd[2]; // Descripteurs de fichier : fd[0] pour lire, fd[1] pour  crire
        pid_t pid;
        char message[] = "Bonjour, fils. Je suis ton p re !\n";
        char buffer[100];

        // Cr er le tube
        if (pipe(fd) == -1) {
                perror(" chec  de pipe");
                exit(1);
        }

        // Cr er le processus
        pid = fork();
        if (pid < 0) {
                perror(" chec  de fork");
                exit(1);
        }

        if (pid > 0) { // Processus p re
                close(fd[0]); // Fermer l  extrmit  de lecture
                write(fd[1], message, strlen(message) + 1); //  crire  le message
                close(fd[1]); // Fermer l  extrmit d  criture
        } else { // Processus fils
                close(fd[1]); // Fermer l  extrmit d  criture
                read(fd[0], buffer,
```

```
                                                sizeof(buffer)); // Lire le message
                                        printf("Fils a re u : %s", buffer);
                                        close(fd[0]); // Fermer l extrmit de lecture
                                }

                                return 0;
                        }
```

- **Explication :**
  - `pipe(fd)` crée deux extrémités : `fd[0]` pour lire, `fd[1]` pour écrire.
  - Le père écrit le message et ferme ses extrémités ; le fils lit et affiche.
  - Fermer les extrémités inutilisées garantit le bon fonctionnement du tube.

Question 2 : Communication bidirectionnelle avec les tubes anonymes

Problème : Écrire un programme C qui permet une communication bidirectionnelle entre un processus père et son fils via des tubes anonymes. Les processus A et B s'échangent une chaîne de caractères : A envoie "hello, je suis le processus A", et B répond par "hello, je suis le processus B". **Solution :**

- **Démarche** : Utiliser deux tubes (un pour chaque direction), `fork()` pour créer un fils, et `write()`/`read()` pour communiquer.
- **Programme :**

```
                        #include <stdio.h>
                        #include <unistd.h>
                        #include <stdlib.h>
                        #include <string.h>

                        int main() {
                                int fd1[2], fd2[2]; // fd1 pour A vers B, fd2 pour B vers A
                                pid_t pid;
                                char buffer[100];

                                // Cr er deux tubes
                                if (pipe(fd1) == -1 || pipe(fd2) == -1) {
                                        perror(" chec  de pipe");
                                        exit(1);
                                }

```

```
pid = fork();
if (pid < 0) {
        perror(" chec  de fork ");
        exit(1);
}

if (pid > 0) { // P re ( Processus A)
        close(fd1[0]); // Fermer l extrmit de lecture de fd1
        close(fd2[1]); // Fermer l extrmit d criture de fd2
        char msgA[] = "hello, je suis le processus A\n";
        write(fd1[1], msgA, strlen(msgA) + 1); // Envoyer B
        read(fd2[0], buffer, sizeof(buffer)); // Recevoir de B
        printf("A a re u : %s ", buffer);
        close(fd1[1]); // Fermer l extrmit d criture
        close(fd2[0]); // Fermer l extrmit de lecture
} else { // Fils (Processus B)
        close(fd1[1]); // Fermer l extrmit d criture de fd1
        close(fd2[0]); // Fermer l extrmit de lecture de fd2
        char msgB[] = "hello, je suis le processus B\n";
        read(fd1[0], buffer,
```

```
                                                sizeof(buffer)); //
                                                 Recevoir de A
                                        printf("B a re u : %s
                                                ", buffer);
                                        write(fd2[1], msgB,
                                                strlen(msgB) + 1);
                                                // Envoyer    A
                                        close(fd1[0]); //
                                                Fermer
                                                   l   extrmit    de
                                                lecture
                                        close(fd2[1]); //
                                                Fermer
                                                   l   extrmit
                                                d  criture
                                }

                                return 0;
                        }
```

- **Explication :**
  - Deux tubes permettent les deux directions : `fd1` pour A vers B, `fd2` pour B vers A.
  - Le père (A) envoie d'abord, le fils (B) répond, avec une fermeture appropriée des extrémités inutilisées.

Question 3 : Application client/serveur

Problème : Écrire deux programmes C :

- Un serveur écrit des messages de 64 caractères dans un tube nommé.
- Un client lit sur ce tube nommé des messages de 64 caractères et les affiche sur la sortie standard.

Solution :

- **Programme Serveur :**

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main() {
        int fd;
        char *fifo = "monfifo";
        mkfifo(fifo, 0666); // Cr er
           un tube nomm
        char message[64] = "Message du
            serveur de 64 caract res
           ...........\n";
```

```

        fd = open(fifo, O_WRONLY);
        write(fd, message, sizeof(message));
        close(fd);
        unlink(fifo); // Supprimer le tube
        return 0;
}
```

- **Programme Client :**

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main() {
        int fd;
        char *fifo = "monfifo";
        char buffer[64];

        fd = open(fifo, O_RDONLY);
        read(fd, buffer, sizeof(buffer));
        printf("Client a re u : %s", buffer);
        close(fd);
        return 0;
}
```

- **Explication :**
  - `mkfifo` crée un tube nommé ; le serveur écrit, le client lit.
  - Les messages sont fixés à 64 caractères, et le tube est supprimé après usage.

Question 4 : Écrire un programme C qui crée trois processus fils : l'un affiche les entiers de 1 à 30, le deuxième de 31 à 60 et le troisième de 61 à 90. Assurez-vous que l'affichage soit ordonné de 1 à 90 en utilisant les signaux (SIGUSR1 et SIGUSR2).

Solution :

- **Démarche** : Utiliser des signaux pour synchroniser. Le père envoie SIGUSR1 pour démarrer chaque fils dans l'ordre, et les fils signalent en retour lorsqu'ils ont terminé.
- **Programme :**

```c
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void handler(int sig) {}

int main() {
        pid_t pids[3];
        int i;

        // Cr er trois fils
        for (i = 0; i < 3; i++) {
                pids[i] = fork();
                if (pids[i] == 0) {
                        signal(SIGUSR1, handler); // D finir le gestionnaire de signal
                        pause(); // Attendre le signal
                        if (i == 0) for (int j = 1; j <= 30; j++) printf("%d\n", j);
                        else if (i == 1) for (int j = 31; j <= 60; j++) printf("%d\n", j);
                        else for (int j = 61; j <= 90; j++) printf("%d\n", j);
                        kill(getppid(), SIGUSR2); // Signaler le p re quand termin
                        exit(0);
```

```
                        }
                }

                // Le p re envoie les signaux dans l ordre
                for (i = 0; i < 3; i++) {
                        kill(pids[i], SIGUSR1); // D marrer le fils
                        pause(); // Attendre que le fils termine
                }

                for (i = 0; i < 3; i++) wait(NULL); // Attendre tous les fils
                return 0;
        }
```

- **Explication :**
  - Les fils attendent SIGUSR1, affichent leur plage, et envoient SIGUSR2 en retour.
  - Le père les déclenche séquentiellement, assurant l'ordre 1-90.

Question 5 : Écrire un programme C qui effectue une communication entre deux processus par une file de messages de clé 17.

Solution :

- **Démarche** : Utiliser une file de messages System V avec `msgget()`, `msgsnd()`, et `msgrcv()`. Un processus envoie un message, l'autre le reçoit.
- **Programme :**

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

// Structure du message
struct msg_buffer {
        long msg_type;
        char msg_text[100];
};

int main() {
        key_t key = 17; // Cl  de la file de messages
```

```
int msgid;
struct msg_buffer message;
pid_t pid;

// Cr er la file de messages
msgid = msgget(key, 0666 | IPC_CREAT);
if (msgid == -1) {
        perror(" chec  de msgget");
        exit(1);
}

pid = fork();
if (pid < 0) {
        perror(" chec  de fork");
        exit(1);
}

if (pid > 0) { // P re (envoie)
        message.msg_type = 1;
        strcpy(message.msg_text, "Message du p re via file de messages\n");
        msgsnd(msgid, &message, sizeof(message.msg_text), 0);
        printf("P re a envoy  : %s", message.msg_text);
} else { // Fils (re oit)
        msgrcv(msgid, &message, sizeof(message.msg_text), 1, 0);
        printf("Fils a re u : %s", message.msg_text);
}

// Supprimer la file de messages (par le p re)
if (pid > 0) {
        wait(NULL); // Attendre que le fils termine
        msgctl(msgid, IPC_RMID, NULL);
}
```

```

                                        return 0;
                                }
```

- **Explication :**
  - `msgget(17, 0666 | IPC`$_CREAT)creunefiledemessagesaveclacl17.Lepreenvoieunmessage$
  - `msgctl` supprime la file après usage.

Question 6 : Qu'est-ce qu'un sémaphore, et comment fonctionne-t-il ?

Solution :

- Un **sémaphore** est comme un compteur qui contrôle l'accès aux ressources partagées.
- **Deux types :**
  1. **Sémaphore binaire** : Comme un verrou—valeur 0 (verrouillé) ou 1 (déverrouillé).
  2. **Sémaphore comptant** : Gère plusieurs ressources—valeur est le nombre de ressources disponibles (par exemple, 3 pour 3 imprimantes).
- **Fonctionnement :**
  - Un processus attend si le sémaphore est 0 (aucune ressource).
  - Si le sémaphore est >0, le processus en prend une (diminue le compteur) et procède.
  - Quand il a fini, il augmente le compteur (libère la ressource).
- **Exemple** : 2 imprimantes (sémaphore = 2). Le processus A en prend une (sémaphore = 1), le processus B en prend une (sémaphore = 0), le processus C attend jusqu'à ce que A ou B termine.

Question 7 : Résoudre le problème Producteur-Consommateur en utilisant un sémaphore.

Solution :

- **Problème** : Un producteur crée des éléments (par exemple, des messages), et un consommateur les utilise. Ils partagent un tampon (stockage) de taille fixe (par exemple, 5 emplacements). Le producteur ne doit pas surcharger, et le consommateur ne doit pas prendre dans un tampon vide.
- **Solution avec sémaphores :**
  1. Utiliser deux sémaphores :
     - **full** : Compte les emplacements remplis (commence à 0).

  - **empty** : Compte les emplacements vides (commence à 5 pour un tampon de taille 5).
2. **Producteur :**
  - Attend si empty = 0 (tampon plein).
  - Ajoute un élément, diminue empty, augmente full.
3. **Consommateur :**
  - Attend si full = 0 (tampon vide).
  - Prend un élément, diminue full, augmente empty.

- **Exemple** : Taille du tampon = 5. Le producteur ajoute 3 éléments (empty = 2, full = 3). Le consommateur prend 2 éléments (empty = 4, full = 1). Ils continuent à coordonner sans erreurs.

TD5 : Communications Inter-Processus (IPC) & Blocages

Résumé : Les communications inter-processus (IPC) permettent aux processus d'échanger des données via mémoire partagée, messages, tubes, signaux ou sockets. Un blocage (deadlock) survient lorsque des processus s'attendent mutuellement en boucle, nécessitant quatre conditions : exclusion mutuelle, attente avec possession, non-préemption et attente circulaire. Les solutions incluent l'ignorance, la détection, l'évitement (ex. : algorithme du banquier) ou la prévention. Ce TD analyse les graphes d'allocation de ressources et l'algorithme du banquier.

Question 1 : Graphe d'allocation de ressources

Problème : État des processus et ressources :

- A détient R, demande S.
- B demande T.
- C demande S.
- D détient U, demande S et T.
- E détient T, demande V.
- F détient W, demande S.
- G détient V, demande U.

Construisez le graphe d'allocation. Y a-t-il un blocage ? Si oui, quels processus sont impliqués ?

Solution :

- **Graphe :**
  - **Nœuds** : Processus (A, B, C, D, E, F, G), Ressources (R, S, T, U, V, W).
  - **Arcs** :

    * A → R (détient), A → S (demande).
    * B → T (demande).
    * C → S (demande).
    * D → U (détient), D → S, T (demande).
    * E → T (détient), E → V (demande).
    * F → W (détient), F → S (demande).
    * G → V (détient), G → U (demande).

- **Blocage :** Recherche de cycles.
  - Cycle : D → T, E → T (détient), E → V, G → V (détient), G → U, D → U (détient).
  - Cycle détecté : D-E-G-D.
- **Résultat :** Blocage impliquant **D, E, G**.

Question 2 : Graphe d'allocation avec 4 processus

Problème : Système avec 4 processus (P1 à P4) et 3 ressources (R1 : 3, R2 : 2, R3 : 2) :

- P1 détient 1 R1, demande 1 R2.
- P2 détient 2 R2, demande 1 R1, 1 R3.
- P3 détient 1 R1, demande 1 R2.
- P4 détient 2 R3, demande 1 R1.

Construisez le graphe. Y a-t-il un blocage ? Si oui, quels processus ?
Solution :

- **Graphe :**
  - **Nœuds** : P1, P2, P3, P4, R1 (3), R2 (2), R3 (2).
  - **Arcs** :
    * P1 → R1 (détient 1), P1 → R2 (demande 1).
    * P2 → R2 (détient 2), P2 → R1, R3 (demande 1).
    * P3 → R1 (détient 1), P3 → R2 (demande 1).
    * P4 → R3 (détient 2), P4 → R1 (demande 1).
- **Blocage :**
  - Cycle : P1 → R2, P2 → R2 (détient), P2 → R1, P3 → R1 (détient), P3 → R2.
  - Cycle détecté : P1-P2-P3-P1.
- **Résultat :** Blocage impliquant **P1, P2, P3**.

Question 3 : Algorithme du banquier

Problème : Système avec 3 processus (P1, P2, P3) et 4 ressources (R1 : 4, R2 : 2, R3 : 3, R4 : 1). État à t_0 :

- P1 détient 1 R3.
- P2 détient 2 R1, 1 R4.
- P3 détient 1 R2, 2 R3.
- Demandes : P1 (2 R1, 1 R4), P2 (1 R1, 1 R3), P3 (2 R1, 1 R2).

L'état est-il sûr ? Peut-on accorder 2 R1 à P3 ? 1 R1 à P2 ?

Solution :

- **État initial :**
  - **Allocation** : P1 [0,0,1,0], P2 [2,0,0,1], P3 [0,1,2,0].
  - **Disponible** : [4-2, 2-1, 3-3, 1-1] = [2,1,0,0].
  - **Max** (hypothèse) : P1 [2,0,1,1], P2 [3,0,1,1], P3 [2,2,2,0].
  - **Besoin** : P1 [2,0,0,1], P2 [1,0,1,0], P3 [2,1,0,0].
- **Sécurité :**
  - Disponible : [2,1,0,0].
  - P1 : Besoin [2,0,0,1] $\not\leq$ A (R4 insuffisant).
  - P2 : Besoin [1,0,1,0] $\not\leq$ A (R3 insuffisant).
  - P3 : Besoin [2,1,0,0] $\not\leq$ A.
  - Aucun processus ne peut terminer. **État non sûr**.
- **Accorder 2 R1 à P3 :**
  - Disponible : [0,1,0,0].
  - P3 Alloc : [2,1,2,0], Besoin : [0,1,0,0].
  - Toujours non sûr.
- **Accorder 1 R1 à P2 :**
  - Disponible : [1,1,0,0].
  - P2 Alloc : [3,0,0,1], Besoin : [0,0,1,0].
  - Toujours non sûr.
- **Conclusion :** État non sûr. Aucune demande ne peut être accordée.

Question 4 : Méthodes de communication IPC

Solution :

- **Mémoire partagée :**
  - Les processus lisent/écrivent dans une mémoire commune.
  - Exemple : Un processus écrit une valeur dans un compteur partagé, un autre la lit.
  - Problème : Nécessite une synchronisation (ex. : verrou).
- **Tubes (Pipes) :**
  - Canal de données unidirectionnel entre processus.
  - Exemple : `ls | wc -l`, où `ls` envoie des données à `wc -l`.
  - Types : Anonymes (parent-enfant) ou nommés (processus indépendants).

Question 5 : Blocage et prévention

Solution :

- **Définition :** Un blocage survient lorsque des processus s'attendent indéfiniment en boucle.
- **Exemple :** A détient R1, attend R2 ; B détient R2, attend R1.
- **Conditions nécessaires :**
  1. **Exclusion mutuelle** : Une ressource est exclusive à un processus.
  2. **Attente avec possession** : Un processus détenant une ressource peut en demander d'autres.
  3. **Non-préemption** : Les ressources ne peuvent être retirées.
  4. **Attente circulaire** : Boucle d'attente entre processus.
- **Prévention :** Éliminer une condition :
  - **Exclusion mutuelle** : Partager les ressources (ex. : spouler les impressions).
  - **Attente avec possession** : Demander toutes les ressources au départ.
  - **Non-préemption** : Permettre la préemption (ex. : mémoire).
  - **Attente circulaire** : Imposer un ordre de demande (ex. : R1 avant R2).
- **Exemple :** Pour deux imprimantes, imposer un ordre (Imprimante 1, puis 2) évite l'attente circulaire.